Smart Moisture Advisor An AI Driven Irrigation Advisory System for Sugarcane

Major Project Report 2026-27
Department of Computer Engineering
K. J. Somaiya Institute of Technology

Students: Kartik Verma and Kushal Soni
Project Guide: Dr. Nilesh Yadav

1 Introduction

Smart Moisture Advisor (SMA) is a browser-based decision-support prototype for sugarcane irrigation. It combines plot attributes with user-selected weather scenarios to estimate when irrigation is due, the water volume and pump duration required, and the consequences of delay. The implemented system covers 1,000 geofenced plots in Mandya district, Karnataka, across six taluks and ten villages [10].

Irrigation planning requires a balance between crop demand, available soil water, expected rainfall, and the practical availability of pumps and electricity. FAO-56 supplies the crop-coefficient and root-zone water-balance framework used to reason about these quantities [1]. SMA connects this agronomic calculation to a plot-level interface that also supports fertiliser advice, risk estimates, explanations, and scheduling.

Project approach

The supplied dataset contains vegetation indices, soil properties, seasonal climate, and plot geometry, but no recorded irrigation dates, irrigation volumes, harvest yields, or disease outcomes. The training pipeline therefore expands each plot into twelve seasonal scenarios and generates target values from agronomic equations with added noise. Seven tree-ensemble models learn these relationships, while K-Means partitions the observed plots into four management zones [10].

Python and scikit-learn handle offline training. Exported JSON trees support inference directly in Node.js, so the deployed web application does not need a Python prediction service. Express serves the API and static client. Leaflet maps the plots, and Chart.js displays predictions and evaluation outputs. English, Kannada, Hindi, and Marathi advisories share the same structured predictions [6], [8]-[10].

Principal outcome and interpretation

The saved held-out-plot evaluation records an irrigation-timing R-squared of 0.9646 and a mean absolute error of 0.6942 days. These results quantify agreement with generated agronomic labels. They do not establish field accuracy or measured water savings. The project demonstrates an integrated software workflow whose next requirement is validation against independent irrigation and harvest records [10].

2 Literature Survey and Research Gaps

Agronomic water-balance methods

Allen et al. describe reference evapotranspiration, crop coefficients, available root-zone water, and soil-water depletion in FAO-56 [1]. This provides a physical basis for deciding when the crop approaches a water-stress threshold. The framework remains dependent on appropriate weather, crop, and soil parameters. SMA adopts its crop-water and depletion concepts while using simplified parameter estimates suited to the available inputs. These simplifications require local calibration before operational use.

Doorenbos and Kassam relate relative yield reduction to relative evapotranspiration deficit using a crop yield-response factor [2]. Their model motivates the project's simulated loss from delayed irrigation. A water-response relationship alone does not identify a farmer's actual yield or allocate competing pumps. SMA adds a delay input and scheduling workflow, but its yield and delay-loss labels still inherit the assumptions of its agronomic generator.

Connected precision irrigation platforms

Kamienski et al. present SWAMP, an IoT platform for precision irrigation with multiple agricultural pilots and a performance study of its FIWARE infrastructure [3]. The paper demonstrates the engineering work required to connect sensing, water management, and scalable services. It also reports that scalability depends on platform configuration. SMA takes a smaller local architecture: a CSV-backed API, portable models, and a browser interface. This reduces prototype infrastructure requirements, while leaving live sensing and operational integration for future work.

The comparison is architectural rather than experimental. SWAMP reports deployments of an IoT platform, whereas SMA currently uses a static plot dataset and manually selected forecast scenarios. No shared benchmark or head-to-head experiment exists in the repository, so a superiority claim would be unsupported [3], [10].

Tree ensembles for tabular prediction

Breiman's Random Forest combines randomized tree predictors through aggregation [4]. SMA uses forests for irrigation timing and the two ordered risk classifications. The spread of individual irrigation trees provides a descriptive uncertainty signal. It is not a calibrated confidence interval. The model's performance depends on how representative the generated scenarios are of the conditions encountered during use.

Friedman's gradient boosting fits an additive model through successive improvements to a loss function [5]. SMA uses gradient-boosted trees for water requirement, irrigation depth, cane yield, and delay-related yield loss. These methods suit nonlinear relationships in numerical inputs. Their flexibility does not remove the need for independent targets, appropriate data splitting, and deployment monitoring.

Research implication

The literature supplies the physical equations, ensemble methods, and examples of irrigation platforms. SMA's contribution is their implementation in one plot-advisory workflow. The remaining research gap is evidence that this combined workflow produces reliable decisions and benefits under actual field conditions [1]-[5], [10].

3 Literature Survey and Research Gaps

Evaluation and reproducible implementation

Pedregosa et al. describe scikit-learn as a consistent Python interface for supervised and unsupervised learning [6]. SMA uses it to train the ensembles and management zones. The GroupShuffleSplit API separates groups rather than individual samples [7]. In this project the group is the plot identifier, so seasonal scenarios from a test plot do not appear in the training partition. This supports a more meaningful test than randomly splitting all scenario rows.

The split still uses the same scenario and label generator on both sides. In addition, village-median NDVI imputation precedes the group split in ml/train.py. Consequently, the evaluation should be described as a grouped prototype benchmark rather than a fully isolated assessment of every preprocessing operation. Fitting preprocessing only on training plots would strengthen a future experiment [10].

Gaps and the implemented response

Gap or constraint	SMA implementation	Remaining evidence needed
No observed target outcomes	FAO-based labels plus noisy seasonal scenarios	Independent irrigation, yield, and disease labels
Advice spans separate operational tasks	One API links prediction, hydraulics, fertigation, and scheduling	Farmer usability and agronomist review
Electricity windows constrain execution	Parallel-lane WSPT scheduling with a first-come comparison	Measured feeder and pump behaviour
Users need reasons and local language	Median-substitution explanations and four language templates	Explanation and translation validation

The table identifies gaps relative to the project's operational requirements. It does not assert that every cited platform lacks these capabilities. The source code supports the listed SMA response, while the final column describes work that the present implementation has not established [10].

Uncertainty and generalisation

The saved additional analysis reports 42.7% coverage within one forest standard deviation and 77.4% within two standard deviations for irrigation timing. These values do not support interpreting the displayed spread as a conventional 68% or 95% interval. The six taluks also have different held-out sample counts and errors, so a pooled score cannot establish uniform reliability across locations [10].

Crop age is a stable hash-derived estimate when no user override is supplied. Forecast rain is a scenario input, and the chatbot detects keywords rather than using a language model. The project has no demonstrated live satellite ingestion, weather-feed connector, pump control, or agronomic diagnosis service. Clear boundaries are necessary when interpreting the integrated interface [10].

4 Problem Definition Objectives and Scope

Problem definition

Develop a plot-specific sugarcane advisory system that converts available vegetation, soil, and climate attributes into understandable irrigation guidance, then helps allocate limited pumping capacity. The system must expose relevant assumptions and allow a person to review the recommendation. Its current dataset and generated labels constrain the claims that can be made about agronomic performance [10].

Objectives

- Prepare a usable plot dataset by parsing geofences, imputing missing NDVI, and flagging unusual values within villages.
- Estimate irrigation timing, crop water requirement, net irrigation depth, yield, delay-related yield loss, water stress, and disease or pest risk.
- Convert model outputs into method-specific gross water volumes, pump hours, rainfall-adjusted advice, and stage-wise fertigation quantities.
- Provide maps, input simulation, model diagnostics, and advisory text in English, Kannada, Hindi, and Marathi.
- Schedule irrigation jobs within power windows and feeder capacity, compare the heuristic with first-come rotation, and support accept or override feedback.

Functional scope

The implemented application supports 1,000 CSV-backed plots, four irrigation methods, seven predictive models, and four management zones. The Farm Advisor uses a plot ID and optionally a revised crop age. The simulator accepts bounded numerical inputs, while the scheduler takes geographic filters, power windows, pump capacity, and a planning horizon. Feedback appends to a local JSONL file [10].

Operating assumptions

Irrigation efficiencies are configured as 0.55 for flood, 0.65 for furrow, 0.75 for sprinkler, and 0.90 for drip. Pump flow defaults to 50 cubic metres per hour. Water and energy comparisons use 70 mm of gross irrigation every eight days and a 3.73 kW pump. These are application parameters rather than measurements of each farm's equipment [10].

Scope boundaries and acceptance

The prototype provides decision support without actuating pumps. Sensor procurement, live forecast ingestion, authenticated multi-user operation, production databases, and measured field outcomes remain outside the implemented scope. Acceptance for this submission concerns an operational local application, traceable model artifacts, consistent Python-to-JavaScript predictions, and readable outputs. It does not imply agronomic deployment readiness.

The existing 23 tests pass in the reviewed checkout. They cover API behaviour, scenario effects, multilingual advisory generation, scheduler constraints, and inference parity with saved Python fixtures. Passing software tests demonstrates the tested behaviour and cannot replace a controlled field trial [10].

5 System Architecture and Data Flow

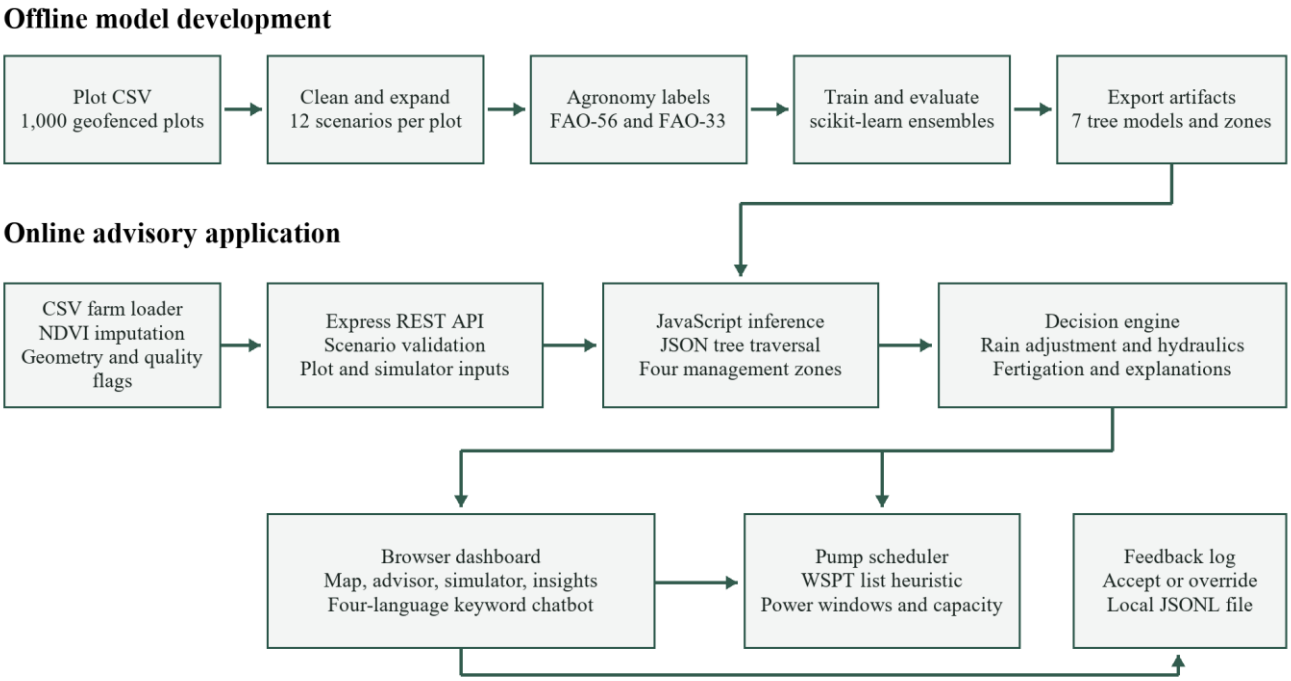


Figure 1. Architecture derived from ml/train.py, src/app.js, src/data/farms.js, src/ml/, and src/engine/. Model training is offline. The running Node.js service reads the CSV and exported models; it does not retrain during requests [10].

Offline preparation and training

The Python pipeline renames the dataset columns, fills six missing NDVI values using village medians, and creates 12,000 weather and crop-age scenarios. Agronomic functions generate labels with stochastic noise. An 80/20 GroupShuffleSplit separates 800 training plots from 200 test plots, producing 9,600 and 2,400 scenario rows. The yield-loss model uses three delay replications, giving 36,000 rows in total [7], [10].

Online prediction and delivery

At startup the farm loader parses plot polygons, computes centroids, estimates crop age, and records data-quality flags. The model registry loads JSON trees and zone centroids. Express validates API inputs, applies scenario settings, and calls the advisory engine. The browser requests summaries, plot details, simulations, schedules, or chat answers and renders the returned data [8]-[10].

Advice and feedback path

The engine combines predictions with irrigation efficiency, pump flow, and growth stage. Separate modules provide feature-sensitivity explanations, language templates, and pump scheduling. Accepting or overriding an advisory records a local feedback entry. The current training pipeline does not automatically consume this log, so feedback collection and automatic model improvement are separate capabilities [10].

6 Implementation and Important Algorithms

Models and targets

Target	Implemented model	Output
Next irrigation	Random Forest	Days and tree spread
Crop water requirement	Gradient Boosting	ETc in mm/day
Net irrigation depth	Gradient Boosting	Millimetres
Cane yield	Gradient Boosting	Tonnes per hectare
Loss from irrigation delay	Gradient Boosting	Percentage loss
Water stress	Random Forest classifier	Four ordered classes
Disease and pest risk	Random Forest classifier	Three ordered classes

A separate K-Means model assigns four management zones from standardised observed features. It is unsupervised and has no accuracy value comparable to the seven supervised targets [10].

Agronomic computation

The label generator estimates reference evapotranspiration with a Hargreaves-style equation and an assumed humidity-dependent temperature range. It blends an age-based sugarcane coefficient with an NDVI-based coefficient to obtain ETc = Kc x ET0. Root-zone depletion is Dr = 1000 x (FC - soil moisture) x root depth. Here FC is field capacity and RAW is readily available water. Days to irrigation equal (RAW - Dr + effective rain) / ETc, bounded to 0-15 days [1], [10].

Effective forecast rain is max(0, 0.8 x (rain - 3)). Net irrigation depth divided by method efficiency gives gross depth. Volume in cubic metres equals gross millimetres x area in hectares x 10; dividing by pump flow gives operating hours. The FAO-33 yield-response concept supports the simulated loss curve, supplemented by project-specific stage and heat factors [2], [10].

Inference and explanation

The JavaScript evaluator traverses exported tree arrays and casts numerical features to float32 to match scikit-learn. Ordered classifiers use the first class whose cumulative probability reaches 0.5. Explanations replace one input with its regional median and report the resulting change in irrigation timing. These are sensitivity effects and should not be described as SHAP values or causal effects [10].

Scheduling and workflow

The scheduler orders jobs by weight divided by pumping duration. Weight combines urgency, predicted tonnes at risk, and stress probability. It packs jobs into parallel feeder lanes, pauses at power-window boundaries, and compares the schedule with first-come rotation. This is a heuristic for the implemented constrained problem, with no global optimality guarantee. Node.js serves the saved models through REST endpoints; core application logic requires no change to produce the report or demonstration [8], [10].

7 Application Outputs and Scenario Results

The following are crops of existing application screenshots. Values describe the selected summer-dry or rainfall scenario, rather than observed farm outcomes. Full screens and saved scenario records are retained in the repository [10].

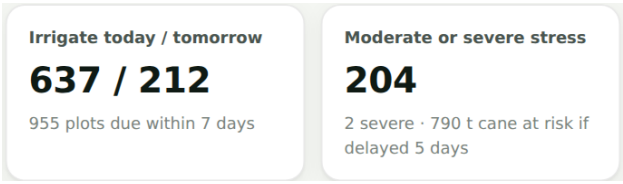


Figure 2 Regional dry-spell overview

The dry-spell screen reports 637 plots due today and 204 with moderate or severe stress. Regional counts summarise the selected seven-day dry and +8 degree temperature scenario.

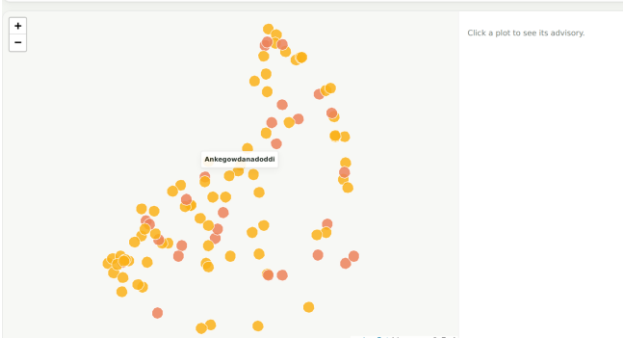


Figure 3 Plot map and management layers

The Maddur map displays geofenced plots coloured by water-stress class. Layer choices also expose irrigation timing, vegetation, moisture, risk, and management zones.

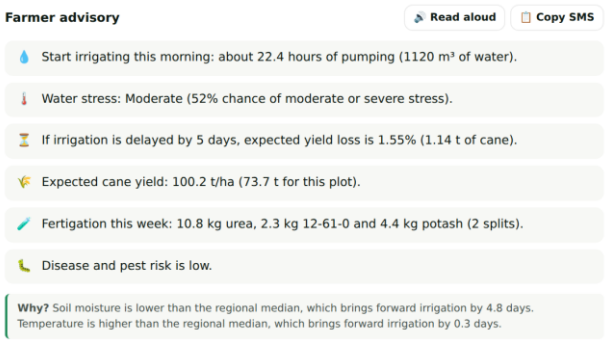


Figure 4 Plot advisory and agronomic outputs

MM-MD-0110 receives 22.4 pump hours and 1,120 cubic metres. Advice includes 100.2 t/ha yield, 1.55% five-day delay loss, fertigation doses, and low disease risk.

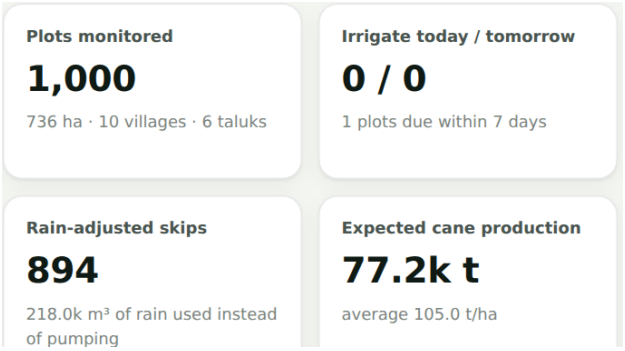


Figure 5 Rain-aware regional advice

With a 40 mm forecast, 894 irrigations receive the rain-skip status and none remain due today. These are model-based scenario counts, with estimated rather than measured savings.

Interpretation of the displayed results

The plot advisory combines irrigation, stress, yield, delay loss, fertigation, and disease-risk outputs. Its explanatory text identifies lower soil moisture as the largest factor bringing irrigation forward. The map provides spatial context for those predictions. The rainfall comparison illustrates counterfactual advice under a changed forecast; it does not demonstrate a realised reduction in water extraction.

The saved advisory screenshot displays a target date of 30 September 2026. This is the interface date in the retained image, rather than an experimental observation date. A separate demonstration records the current local application.

8 Evaluation and Further Application Outputs

Table 2. Saved test results on generated labels for held-out plots. Regression scores use R-squared and MAE in the stated units; classification scores use accuracy and macro-F1. Values are read from models/metrics.json [10].

Target	R-squared / acc.	MAE / macro-F1	Baseline score
Irrigation timing (days)	0.9646	0.6942	0.7444
Water need (mm/day)	0.9768	0.1689	0.8843
Net depth (mm)	0.9745	1.7471	0.6206
Yield (t/ha)	0.8918	3.3550	0.5709
Delay loss (percentage points)	0.9755	0.3566	0.4990
Water stress	87.83%	0.8172	83.46%
Disease and pest risk	87.29%	0.7627	81.96%

All seven ensemble scores exceed their saved linear or logistic baseline. Timing MAE falls from 2.5796 to 0.6942 days. These comparisons assess approximation of the label generator and do not quantify field accuracy.

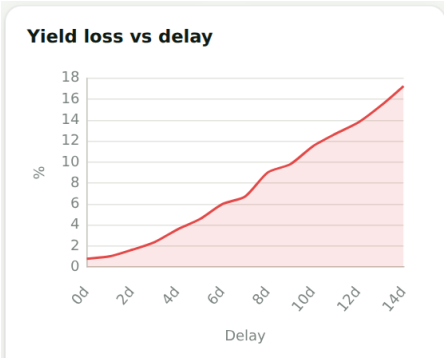


Figure 6 What-if simulation

The heat-wave preset shows severe water stress and recalculates irrigation need. Its delay curve and feature effects allow the user to inspect sensitivity to changed conditions.



Figure 7 Power-window scheduling

For Hosur, 20 pumps and a three-day horizon, the saved run completes 8 plots versus 5 under rotation. Estimated cane loss is 80.23 t versus 85.60 t.



Figure 8 Kannada advisory

The same structured recommendation appears in Kannada. English, Hindi, and Marathi are also supported, with read-aloud dependent on browser voice availability.

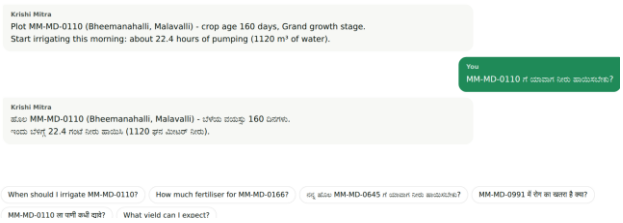


Figure 9 Krishi Mitra conversation

English and Kannada questions retrieve the plot advisory. Keyword matching recognises irrigation intent and the plot ID, without an external language-model service.

Model Insights also provides feature importance, predicted-versus-label plots, confusion matrices, and data-quality summaries. The 23 passing software tests include parity against saved Python fixtures and scheduling constraints. Neither these diagnostics nor the saved scenario comparisons constitute a field trial [10].

9 Conclusion and Future Scope

Conclusion

SMA implements a complete local software path from geofenced plot data to irrigation advice and pump scheduling. Its browser interface presents model predictions alongside hydraulic conversions, rainfall scenarios, stage-wise fertigation, language templates, and feature-sensitivity explanations. Python-to-JSON export allows the Node.js application to serve the trained tree ensembles without a separate Python prediction service [10].

The saved evaluation shows that the ensembles approximate the project's generated targets better than their linear or logistic baselines. Irrigation timing reaches an R-squared of 0.9646 with 0.6942 days MAE, while the classifiers obtain 87.83% water-stress accuracy and 87.29% disease-risk accuracy. Existing tests confirm the tested API, model-serving, language, and scheduling behaviour. The results demonstrate a functioning prototype and reproducible computational workflow [10].

Limitations

No independent field labels validate irrigation timing, yield, or disease risk. The input data are a static regional snapshot, crop age defaults to an estimate, and weather forecasts are manually selected scenarios. Soil hydraulics, radiation, equipment efficiency, and pump flow include simplifying assumptions. Tree spread is uncalibrated, and the scheduler's estimated benefit depends on the same generated yield-loss relationship that informs its priorities.

The interface can log accept or override feedback, but the training pipeline does not automatically incorporate those records. The local API also lacks a demonstrated production identity and access-control system. Wider deployment therefore requires additional agronomic, operational, and software engineering work [10].

Future scope

- Collect time-stamped irrigation events, measured discharge, soil profiles, disease observations, and harvest yield. Validate by season and location with preprocessing fitted only on training data.
- Replace estimated crop age with planting records and connect verified weather and earth-observation feeds. Maintain observation timestamps and validate the meaning of each input.
- Calibrate uncertainty against independent outcomes and evaluate error separately across taluks, growth stages, and extreme weather. Ask agronomists to review fertigation and risk rules.
- Compare the schedule with stronger optimisation methods using measured feeder capacity, pump discharge, interruptions, and farmer availability. Report actual water and energy use in a controlled pilot.
- Add role-based access, durable data storage, monitored services, and a reviewed feedback-to-retraining workflow. Test the four-language interaction with intended users.

10 References

IEEE-style numbered references. Online sources checked on 19 October 2026. Project implementation, evaluation, and screenshot claims cite [10].

[1] R. G. Allen, L. S. Pereira, D. Raes, and M. Smith, Crop Evapotranspiration: Guidelines for Computing Crop Water Requirements, FAO Irrigation and Drainage Paper 56. Rome: FAO, 1998. https://www.fao.org/4/x0490e/x0490e00.htm

[2] J. Doorenbos and A. H. Kassam, Yield Response to Water, FAO Irrigation and Drainage Paper 33. Rome: FAO, 1979. Bibliographic record: https://www.fao.org/4/y3655e/y3655e03.htm

[3] C. Kamienski et al., "Smart Water Management Platform: IoT-Based Precision Irrigation for Agriculture," Sensors, vol. 19, no. 2, article 276, 2019. doi: 10.3390/s19020276. https://www.mdpi.com/1424-8220/19/2/276

[4] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5-32, 2001. doi: 10.1023/A:1010933404324. https://doi.org/10.1023/A:1010933404324

[5] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001. doi: 10.1214/aos/1013203451. https://doi.org/10.1214/aos/1013203451

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011. https://www.jmlr.org/papers/v12/pedregosa11a.html

[7] Scikit-learn developers, "GroupShuffleSplit," official API documentation. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupShuffleSplit.html

[8] Express contributors, "Express 5.x API," official documentation. https://expressjs.com/en/5x/api.html

[9] Leaflet contributors, "Leaflet API reference," official documentation. https://leafletjs.com/reference.html

[10] K. Verma and K. Soni, Smart Moisture Advisor, project implementation and repository artifacts, Interior-Gardener/SMA, local main checkout reviewed 19 October 2026. https://github.com/Interior-Gardener/SMA. Evidence: ml/train.py, ml/agronomy.py, src/, models/metrics.json, models/test_fixtures.json, report/data/results.json, report/data/extra.json, and presentation/screenshots/.